Mobile Robot Chassis User and Development Manual

Hardware, Operation, Motion Control, Communication and Firmware Development

Technical Edition · V1.2 · 13 October 2026

Contents

Chapters 1–6	Chapters 7–11
1. Introduction	7. Chassis Feedback and Sensor Data
2. Chassis Components and Interfaces	8. Internal Chassis Communication
3. General Operation, Configuration and Debugging	9. Control Architecture and Firmware Tasks
4. Chassis Types and Kinematic Analysis	10. STM32 Firmware Programming
5. Hub-Motor Control	11. Precautions and Maintenance
6. Chassis Command Interfaces	

Preface

This manual describes the hardware, operating procedures and low-level development interfaces for the LSR100, LSR300 Mini, LSR200 Series, LSR300 Pro and LSR280 models. These platforms use the **C63A** controller architecture and share several host interfaces; individual features, motor arrangements, sensor options and software functions vary by model and ordered configuration.

The intended readers are engineers and researchers familiar with robotics, ROS, C programming and basic electrical interfaces. The manual explains the STM32 chassis layer; higher-level ROS/ROS 2 packages, mapping and navigation software must be checked against the actual delivered configuration. In particular, a reference to a supported interface does not mean an optional device, application or source-code package is included in every order.

Safety and version notice. Technical interfaces in this edition apply to the documented controller and firmware configurations dated 13 October 2026. Before connecting third-party hardware, reprogramming firmware or changing parameters, verify the controller revision, pinout, motor type and firmware version installed on the actual robot. Some firmware descriptions require model-specific verification, especially where command flags or engineering units differ between interface sections; these cases are noted explicitly.

Revision history

Version	Date	Summary
V1.0	2024-09-27	Initial release
V1.1	2025-10-22	Updated procedures and functions after firmware revisions
V1.2	2026-10-13	Added LSR280 hardware and control notes

1. Introduction

1.1 Mobile chassis and ROS

The mobile robot system is typically divided into a host decision layer, an environment perception layer and a real-time execution layer. ROS provides the message-passing environment through which navigation, localization and user applications exchange commands and observations. An STM32-based chassis handles low-level command reception, drive control and robot-state acquisition. A ROS node running on a host computer communicates with that controller, usually through a serial link; the bare-metal/RTOS STM32 controller itself is **not** a native ROS graph node.

The main integration functions are **command execution** (chassis velocities to wheel or steering targets), **status feedback** (wheel/odometry, battery and onboard IMU data) and **system integration** with higher-level localization, mapping and autonomy modules. The interfaces described below allow custom applications to drive the chassis without redesigning the low-level drive electronics.

1.2 LiDAR and ROS

A LiDAR reports ranges and, depending on the sensing system, other measurements that help build a map or detect obstacles. A ROS application can combine LiDAR with IMU, wheel odometry and additional sensors for localization, obstacle detection and navigation. These are host-system functions; they should not be confused with the basic motion control implemented on the C63A controller. Whether a LiDAR or a particular mapping package is provided depends on the purchased configuration.

1.3 Chassis coordinate convention

The chassis uses a right-handed planar convention: **+X forward, +Y to the robot's left, and positive yaw about +Z counterclockwise** when viewed from above. This sign convention applies to motion commands, transmitted chassis velocities, and aligned IMU feedback. Differential and non-holonomic 4WD models cannot directly execute independent +Y motion; the LSR280 four-wheel-steer configuration has different mobility.

2. Chassis Components and Interfaces

2.1 System layers and primary controller

The chassis contains a main control board, one or more servo motor drivers, brushless hub motors, a battery, and a configuration-dependent set of peripheral devices. The **C63A controller** uses an **STM32F407VET6** microcontroller to receive host commands and supervise the drive system. It incorporates an **ICM20948** nine-axis IMU, an OLED status display and interfaces for external peripherals. The controller features include: serial ports, CAN, RS485, six ultrasonic-sensor connectors, two programmable RGB LED-strip outputs, an external 5 V power output and optional controller/input interfaces. The newer C63A hardware revision includes a USB/controller interface for a compatible handheld controller.

Check the controller PCB silkscreen and model-specific enclosure wiring drawing for connector pin numbers. General statements about I/O availability are **not** substitutes for a wiring and polarity check on the actual PCB.

2.2 Servo drivers

The covered chassis configurations use brushless servo hub motors driven by external servo drivers. One documented dual-wheel servo-driver model is **ZLAC8015D**, with these reference characteristics:

Parameter	Documented value
Motor driver supply voltage	24–48 V DC
CAN communication	CANopen, CiA 301 / CiA 402; bus rate 100–1000 kbit/s
Operating CAN rate	**500 kbit/s**
RS485 communication	Modbus RTU, rates 9,600–128,000 bit/s
Operating RS485 rate	**115,200 bit/s**
Drive operating modes	Position, velocity and torque control
Driver protection	Overvoltage, overcurrent and other drive-level fault detection

A driver exposes CAN-H/CAN-L and RS485 terminals. The driver uses an eight-pin communications assembly containing two CAN connectors and two RS485 connectors. The correct connector orientation must be established from the physical markings. For differential-drive LSR100/LSR300 Mini/LSR300 Pro chassis, a single dual-channel driver typically drives two hub motors; the LSR200 Series 4WD configuration uses a second driver for four driven wheels.

LSR280 driver configuration

The LSR280 has **four servo drivers**: two for wheel steering and two for wheel propulsion. Each steering driver controls two steering motors in **position mode**, while each propulsion driver controls two hub motors in **velocity mode**. The steering and drive units have the same general enclosure/hardware form but **different firmware**. The wheel designations, facing forward, are **A** = **left rear, B** = **left front, C** = **right front, D** = **right rear**.

CAN driver address	Role	Left output	Right output	Mode
1	Steering	A	B	Position
2	Steering	C	D	Position
3	Hub-wheel propulsion	A	B	Velocity
4	Hub-wheel propulsion	C	D	Velocity

The motor's physical Left and Right terminals are marked on the corresponding driver. Do not interchange motor power wires and encoder/signal leads.

2.3 Brushless hub motors and wheels

Hub motors integrate the motor and wheel assembly to provide compact direct drive and reduced transmission hardware. A typical assembly includes a stator, shaft, magnets, encoder, rim, covers and tire. The motor harness includes separate **power** and **encoder** connections.

Chassis model	Nominal wheel size
LSR100	**5 in**
LSR300 Mini	**5 in**
LSR200 Series	**8 in**
LSR300 Pro	**8 in**

These are **nominal wheel sizes** rather than guaranteed measured outer diameter or tire width. Mechanical integrations requiring exact dimensions should use the unit-specific drawing or measured wheel assembly. The OLED hardware status may identify wheel type as 5-S (5 in, single shaft), 5-D (5 in, dual shaft), 8-S or 8-D.

2.4 Drive wiring and wheel assignment

The baseline drive wiring connects the C63A controller by CAN to one or more servo drivers, and each driver to its wheel motors. For the LSR100/LSR300 Mini/LSR300 Pro differential chassis, two powered wheels connect to one dual-channel servo driver. For the LSR200 Series 4WD chassis, two servo drivers are connected to the CAN network. For the LSR280, four drivers use the address/wheel mapping in §2.2. CAN devices share the bus only when correctly terminated and assigned unique IDs; check the installed cable harness before modifying it.

2.5 RGB status lighting

Depending on model and configuration, LSR100, LSR300 Mini and LSR300 Pro have RGB light strips; LSR280 also supports status lighting. LEDs indicate startup, low battery, charger state and some peripheral events. The firmware supports setting a user-defined color through a serial command described in §6.1, but the function may be limited to installed strips.

2.6 Automatic charging option

An optional automatic charging kit comprises a docking station and a receiver/contact assembly on the robot. The station is supplied from an external power adapter and emits infrared signals used for docking. Electrical charging contacts engage when the robot arrives at the station. Install the dock **against a wall** and keeping the area within approximately **1.5 m** clear of obstructions.

The docking receiver is integrated inside the chassis and is not intended for routine user removal or retrofit. This section describes the **optional** charging kit; it does not imply every Standard chassis includes a docking station. For manual charging and adapter details, consult the delivered model-specific accessory list and battery/charger label.

2.7 Ultrasonic obstacle sensing

Selected models, including LSR300 Mini and LSR300 Pro, have ultrasonic sensors for near-field obstacle sensing. Some outdoor variants of the LSR200 Series also have these sensors. The C63A provides **six ultrasonic inputs**, and an installed sensor uses a **GH2.0 4-pin** connection. The documented sensor has an IP67-class sealed probe, nominal **3 cm near blind zone** and pulse-width-based ranging output.

Pin	Signal	Description
1	VCC	3.3–5 V supply
2	GND	Ground
3	TRIG	Range trigger input
4	ECHO	Echo pulse output

A short trigger pulse with a falling edge initiates a range sample. The returned ECHO pulse width T gives the approximate distance $S = T \times V / 2$, where V is the speed of sound (approximately **348 m/s** at the reference ambient condition). The recommended trigger interval is **greater than 70 ms**; absent an echo, the documented module returns a nominal **35 ms** pulse. The documented timing specifies a trigger pulse *shorter than 10 ms*; this should be checked against the exact installed ultrasonic sensor datasheet before custom firmware implementation.

2.8 Switches and buttons

A typical chassis has a main power button, a motor-disabling software emergency-stop switch and, on some configurations, a power-disconnect hardware emergency-stop switch. **The LSR100 is specifically noted as lacking the hardware emergency power-disconnect switch due to packaging constraints.** Never assume

an E-stop electrically isolates a platform unless this is verified from that unit's schematic. The software stop disables motor control without removing battery power.

The USER button changes OLED pages: **single-click** = **next page; double-click** = **previous page**. The RESET button restarts the STM32 controller. The board contains a model-selection potentiometer, as described below.

2.9 Model-selection potentiometer

The controller firmware can contain configuration data for several supported chassis models. A potentiometer selects the active model/motor parameters; the resulting selection appears on the OLED home screen. After changing the selection, **press RESET** for it to take effect. Changing model selection without matching hardware can cause incorrect motor commands: only use the setting specified for the chassis actually supplied.

3. General Operation, Configuration and Debugging

3.1 Supported model names on the display

The OLED home screen displays internal firmware model codes, which are distinct from the LucSys Robotics commercial model names. The original codes are retained below exactly as shown by the controller so that users can identify the correct configuration:

OLED model code	Corresponding platform
S300	LSR300 Pro mobile robot
S200	LSR200 Series legacy-body 4WD configuration
ODS200	LSR200 Series outdoor 4WD configuration with ultrasonic sensors
MNS300	LSR300 Mini
S100	LSR100 differential-drive service robot
T280	LSR280 four-wheel-steering mobile robot

3.2 Bluetooth application and parameter settings

Supported chassis configurations can be operated and configured by the companion mobile application over Bluetooth. **The actual Bluetooth broadcast name and availability of a compatible app are firmware/product-dependent**; verify both using the documentation delivered with the robot. The documented default Bluetooth pairing PIN is **1234**. A successful connection can be confirmed when live values update in the application's diagnostic or debug display.

The joystick must first be pushed **forward** to unlock app motion control; a higher-priority active source overrides the app. Some apps include a parameter page with a command to read current parameters and another command to save parameters for retention across power cycles.

Parameter	Description	Reference value / warning
0	Default control speed	500 mm/s
1	IMU straight-line correction coefficient	Keep **50**, do not adjust in normal use
2	Hub motor speed-loop Kp	Driver/firmware tuning value
3	Hub motor speed-loop Ki	Driver/firmware tuning value
4	Hub motor current-loop Kp	Driver/firmware tuning value
5	Hub motor current-loop Ki	Driver/firmware tuning value

Motor-loop parameters should be changed conservatively by qualified personnel, with the robot safely supported. The firmware adjustment guide recommends changing in small increments (no more than

approximately 100 display units per step) and issuing a brief motion command to exit parking mode before evaluating the new response.

The application's custom-function buttons may expose: IMU gyro-zero calibration (button 0), auto-docking enable (1), motor fault reset (2), emergency override (3, repeated presses), ultrasonic avoidance enable (4), IMU straight-line assistance (5), OLED previous (6), and OLED next (7). Availability varies with installed hardware. **Emergency override is an abnormal-recovery feature, not a routine control mode.**

3.3 Handheld controller

Where the compatible multimode handheld controller is installed, switch on the handheld device and wait for its link indicator to show a successful connection. The operating sequence uses Start and moving the left stick forward to unlock robot motion. The handheld controller has command and function buttons for setting speed, selected modes and OLED pages; a short onboard beep confirms the accepted function command. These controls are not provided on every chassis or C63A revision.

3.4 Control-source priority and behavior

The firmware arbitrates competing control sources using priority. In the documented implementation the priority order, from highest to lowest, is:

PS2 handheld controller → RC model transmitter → Bluetooth app → ROS/Serial Port 1 → CAN → automatic docking controller.

A higher-priority source interrupts lower-priority commands. The lower-priority source becomes eligible only after the higher-priority source has been inactive for **more than 2 seconds**. The active source appears on the OLED as PS2, R-C, APP, ROS, CAN, or RCM for auto-docking. Software customization can change the priorities, so the supplied firmware must be checked before a safety-critical integration. Some firmware revisions describe CAN mode as exclusive after activation; the priority and CAN-mode behavior should be verified against the installed firmware rather than assumed to be universally identical.

Parking mode

On level ground, after **10 seconds** at rest without command activity, the chassis automatically enters a low-power parking mode. The motor servo loop releases accumulated current/stress before reducing power. Consequently the holding torque may be weaker than during active motion control. A new valid command releases parking mode.

IMU-assisted straight-line driving

IMU straight-line assistance is enabled by default on the documented LSR100, LSR300 Mini, LSR200 Series and LSR300 Pro configurations. This function is not available on the documented LSR280 configuration; steering motor zero calibration can be used to improve tracking. Keep the C63A controller firmly mounted. Disable IMU-assisted correction when diagnosing an abnormal IMU or testing an unmounted controller board.

Ultrasonic obstacle avoidance

On configurations with ultrasonic sensors, obstacle avoidance is **disabled by default**. When enabled, some chassis will turn away if an obstacle is detected ahead and stop when reversing toward a rear obstacle (where rear sensors are fitted). Repeated beeps indicate an avoidance event. This is a basic chassis-assistance function, not a certified personnel-protection system. Disable it when narrow-corridor operation makes reflexive movement undesirable, subject to local risk assessment.

Emergency override mode

The firmware has a maintenance/recovery override that may ignore restrictions related to stop state, battery state and motor faults, allowing a platform with partially functional motors to move to a safe location. **Do not use it near people or as a substitute for diagnosing a fault.** It may bypass protections and therefore should only be used by trained personnel after identifying an appropriate recovery route.

3.5 LED status meanings

Indication	Status meaning
Red/green/blue cycling	Power-up self-test
Steady yellow	Ultrasonic obstacle detected
Slowly flashing purple	Low battery
Steady purple	Auto-docking enabled/searching for dock
Dim red	Docked and charging
Dim green	Charge approaching completion

3.6 Buzzer signals

- **Short beep:** startup or accepted application/handheld function setting.
- **One long beep:** successful nonvolatile-parameter save or emergency override activated.
- **Fast repetitive beeping:** ultrasonic avoidance is actively detecting an obstacle.
- **Approximately 1 Hz periodic beeping:** low battery on platforms without status lighting.

3.7 OLED status screens

The controller OLED shows a different number of pages depending on the selected chassis type. Use USER, the companion application or an installed handheld controller to navigate the screens.

Screen	Content
Home	Model selection, motor/driver values, active control source, battery voltage; LSR280 also displays A/B/C/D steering angles (rad) and wheel speed (mm/s)
Screen 2	Onboard IMU diagnostic information
Screen 3	Auto-docking diagnostic values, including OL (online) status and infrared acquisition flags
Screen 4	Ultrasonic distance readings and timeout counters, where applicable
Screen 5	Handheld/RC channel values LX, LY, RX, RY and enable-state diagnostic information
Screen 6	Selected model and hardware version; motor-driver count and 5-in/8-in hub-motor type
Subsequent screens	Detailed servo-driver online state, CAN link quality and fault codes

C63A hardware **V1.0** as a revision without compatible multimode handheld controller support and **V1.1** as a revision with support. A drive communication-quality value below approximately **70%** suggests checking wiring and excessive CAN cable length. The auto-docking OL flag should be 1 when the docking accessory is online; otherwise inspect its CAN connection if installed.

3.8 Diagnostic modes

The chassis supports serial/application diagnostic logging levels (where the firmware exposes this feature). Commands are ASCII strings:

Command	Diagnostic level
LOG0	Turn diagnostics off

Command	Diagnostic level
LOG1	Basic status information
LOG2	Controller firmware/hardware and auto-docking information
LOG3	Detailed status for controller, peripherals and motor drivers

To collect diagnostics from a serial terminal, connect to **Serial Port 1** or the corresponding external debug connector on configurations fitted with one. A terminal should use the settings specified for that firmware (the documented serial control interface is **115200 baud, 8 data bits, no parity, 1 stop bit**). Send LOG3, then inspect returned text. On some boards the external debug Type-C connector is positioned differently from the C63A on-board ports. Windows diagnostic programs are optional conveniences; a generic serial terminal can carry out the serial part of the procedure.

4. Chassis Types and Kinematic Analysis

4.1 Chassis drive layouts

Type	Example models	Forward/reverse	Lateral motion	Rotation in place	Relative load/steering friction
Differential drive	LSR100, LSR300 Mini, LSR300 Pro	Yes	No	Yes	Lower / lower
4WD skid steering	LSR200 Series	Yes	No	Yes	Higher / higher
Four-wheel steering and drive	LSR280	Yes	Yes	Yes	High loading / low steering friction

The low-level controller converts desired robot velocity into individual wheel and steering targets. **Forward kinematics** estimates chassis velocity from wheel-state measurements; **inverse kinematics** calculates wheel commands from desired body velocity. Differential-drive and skid-steer geometries have different wheelbase/trackwidth parameters and motion constraints from the LSR280 steering layout.

4.2 Kinematic data and integration

The supported chassis categories are defined here; detailed wheel-by-wheel kinematic derivations are provided in model-specific control resources. No complete numeric wheel-separation, axle spacing or LSR280 steering geometry table is contained in this manual. For an application requiring high-accuracy odometry or wheel-level control, request the actual model's dimensional drawing and software configuration. Do not substitute dimensions from another robot model.

For a differential-drive body with effective wheel linear velocities v_L and v_R and track width b, the generic planar relationships are v_x = (v_R + v_L)/2 and omega_z = (v_R - v_L)/b. These relationships explain the referenced STM32 implementation, but model-specific wheel sign, numbering and calibrated dimensions must be verified before implementation.

5. Hub-Motor Control

5.1 Servo-driver closed-loop control

The brushless servo-wheel drive uses an encoder and a servo driver to close the motor speed/current loop. The C63A calculates target wheel commands, configures the drives and receives motor measurements. The documented servo drivers support **CANopen** and **RS485/Modbus RTU**, with **CAN as the default low-level driver bus** for the firmware reference.

5.2 CANopen initialization and functions

Each CANopen servo drive is identified by its node address. The firmware provides a C-level driver-initialization API named:

```
uint8_t ServoDriver_Init(ServoDriverHWType_t *driver);
```

This routine checks whether the drive is online, configures motor and encoder parameters according to the selected wheel type, enables the intended control mode, maps state messages and starts the CANopen network. The example code queries object 0x2031 for software version, accesses motor-target object 0x60FF for speed commands and configures a drive heartbeat period of **5000 ms**. The implementation distinguishes **5-inch/8-inch and single-shaft/dual-shaft** encoder configurations; do not copy its numbers to an unverified motor revision.

The wheel-speed API presented is:

```
int8_t set_MotorRPM(ServoDriverHWType_t *driver,
                    short left_rpm, short right_rpm);
```

The function packs left and right target speeds into one 32-bit value and writes to 0x60FF with subindex 3. Driver status is available using mapped CANopen messages, for example the reference API:

```
ServoDriver_MappingTPDO0(driver,
                         0x190 + driver->NodeID - 1);
```

The firmware documentation refers to returned mapped motor data as encoder/speed information. Note that its description varies between *encoder counts* and *measured RPM*: refer to the drive object dictionary for the unit and scaling of the active mapping.

5.3 RS485/Modbus motor interface

The C63A uses an MCU USART-to-RS485 interface for drive communication. The firmware defines these example functions:

```
void RS485_Hub_Init(void);
void RS485_set_MotorRPM(short left, short right);
```

The RS485 branch uses Modbus RTU frames and CRC checking. In the example, motor-speed output is clamped to ±**3000** internal RPM command units and written to two Modbus registers starting at address 0x2088 using function code 0x10. These are implementation details of the supplied reference firmware, not guaranteed generalized motor limits.

Drive monitoring requires *polling* rather than automatic mapped broadcast messages in the reference RS485 implementation. The documented query APIs are:

```
void RS485_Read_StatusWord(void);
void RS485_Check_MotorError(void);
void RS485_Read_Hub_Temperature(void);
void RS485_Read_Motor_Temperature(void);
void RS485_Read_Motor_RPM(void);
void RS485_Read_Motor_TOR(void);
void RS485_Read_Motor_POS(void);
```

The described RS485 API is supported for **LSR200 Series and LSR300 Pro** in the referenced code version. Serial Port 3 interrupt handling receives the replies. CANopen is generally preferred in the reference because it supports drive telemetry mapping and is the normal configured bus.

5.4 Protection and parameter changes

Drive initializers configure encoder precision, Hall offsets, pole pairs, current limits, speed-loop gains, overload protection, temperature protection and acceleration/deceleration settings. Such settings are **motor- and firmware-specific**. Altering them can destroy motors or reduce safety margins. Save changes only after validating correct wheel types and no-load behavior.

6. Chassis Command Interfaces

6.1 ROS/host serial command protocol

The C63A distinguishes two MCU serial interfaces: **Serial Port 4 / UART4** for the ROS host-computer connection, and **Serial Port 1 / UART1** for general USART/debug/programming use. Both are described as **115200 baud** for chassis commands. Although the two serial-command frame formats are the same, **do not use Serial Port 1 as the normal ROS connection**: incoming host packets at boot can be mistaken for firmware-download traffic and prevent the STM32 from starting correctly.

The host sends an **11-byte** command packet, with the following layout. Signed 16-bit fields are sent **most significant byte first**.

Byte index	Field	Type / meaning
0	Header	0x7B
1	Function selector	0 motion; 1 or 2 motion + auto-docking enable; 4 RGB control
2	Function / operation	Depends on firmware/selection; source also describes link-loss behavior here
3–4	X velocity	Signed 16-bit, **mm/s**
5–6	Y velocity	Signed 16-bit, **mm/s**
7–8	Z/yaw velocity	Signed 16-bit, **rad/s × 1000**
9	Checksum	Bitwise XOR / BCC of bytes **0–8**
10	Footer	0x7D

For RGB control (selector = 4), the relevant payload bytes represent red, green and blue intensity rather than chassis target velocities. The operation byte controls whether custom RGB settings are active.

Reference forward command, +100 mm/s X:

7B 00 00 00 64 00 00 00 00 1F 7D

Reference leftward velocity command, +100 mm/s Y (on a compatible holonomic model):

7B 00 00 00 00 00 64 00 00 1F 7D

Reference yaw command, +0.1 rad/s:

7B 00 00 00 00 00 00 00 64 1F 7D

Reference stop command:

7B 00 00 00 00 00 00 00 00 7B 7D

Issue motion commands continuously at **intervals no greater than 100 ms**. When validating a newly integrated host, support the chassis with all driven wheels off the floor, ensure that no person can approach moving wheels and test a **zero-velocity command first**.

Important ambiguity: link-loss setting. The function-operation byte is reserved for some functions; a firmware example additionally states that 0 selects stop-on-control-loss and 1 selects hold-last-target behavior. Because these descriptions conflict, **do not rely on a particular command byte to provide fail-safe stopping without confirming the active firmware implementation and testing it**.

6.2 CAN chassis command protocol

The host may send a standard **11-bit CAN data frame** through **CAN1 at 500 kbit/s**. The documented motion-control ID is **0x181**, with **DLC = 8**:

Byte	Content
0–1	X target speed, signed int16, mm/s, MSB first
2–3	Y target speed, signed int16, mm/s, MSB first
4–5	Z/yaw target speed, signed int16, rad/s × 1000, MSB first
6–7	Reserved

Use CAN-H to CAN-H and CAN-L to CAN-L, and verify ground/reference and bus termination per the actual wiring diagram. Differential-drive and 4WD skid-steer chassis ignore or cannot realize a nonzero lateral command. A firmware-dependent CAN command source appears as CAN on the OLED when active.

6.3 Mobile application control

The documented Bluetooth application sends commands and receives diagnostic data through the UART2-linked wireless interface; the firmware uses a rate of **9600 baud** for that communication channel. The joystick and function controls select movement, speed and chassis modes. The application uses the following motion codes:

ASCII/hex byte	Action
0x41	Forward
0x42	Forward-right
0x43	Rotate right
0x44	Reverse-right
0x45	Reverse
0x46	Reverse-left
0x47	Rotate left
0x48	Forward-left

Push the stick forward for about **0.5 seconds** after successful Bluetooth connection to enable driving. The behavior of additional keys and modes depends on the app and firmware revision.

6.4 Multimode handheld control

A supported compatible controller can send motion and mode commands after successful pairing and unlocking, as described in §3.3. Not all C63A revisions support the same controller. This manual describes the host protocol but does not ship companion application or handheld software.

6.5 RC transmitter and receiver

A compatible RC transmitter may connect via a receiver attached to the chassis control interface. The operating procedure is: connect the receiver, power on the chassis, put the transmitter's right stick into its

lowest/start position as specified for the transmitter, power on the transmitter, wait for the successful receiver link indicator, then move the left stick forward to initiate RC mode. The OLED shows R-C when active. The left stick controls forward/reverse and the other stick controls rotation/steering; channel assignments may differ with holonomic variants.

The documented receiver-to-adapter mapping is:

Receiver	Adapter terminal
GND	-
5 V	+
CH1	D12
CH2	D13
CH3	D14
CH4	D15 (required only for applicable lateral-motion control)

Match polarity and signal levels to the specific supplied receiver and interface; do not assume the connector layout from the above functional mapping.

7. Chassis Feedback and Sensor Data

7.1 Base-state feedback over serial

The chassis outputs a **24-byte** standard base-state data frame. Signed 16-bit fields use **MSB first**. The fields are arranged as follows:

Byte index	Field	Unit / interpretation
0	Header	0x7B
1	flag_stop / motor enable status	Firmware-dependent flag; see note below
2–3	X velocity	Signed int16, mm/s
4–5	Y velocity	Signed int16, mm/s
6–7	Z/yaw velocity	Signed int16, rad/s × 1000
8–9	Accelerometer X	Signed int16, raw
10–11	Accelerometer Y	Signed int16, raw
12–13	Accelerometer Z	Signed int16, raw
14–15	Gyroscope X	Signed int16, raw
16–17	Gyroscope Y	Signed int16, raw
18–19	Gyroscope Z	Signed int16, raw
20–21	Battery voltage	Signed int16, millivolts
22	BCC checksum	XOR of bytes 0–21
23	Footer	0x7D

Reference frame (hex):

7B 00 00 9B 00 00 FF DF 00 60 00 0C
40 A8 FF FD 00 06 00 1E 5B 87 82 7D

The example yields **X** = **+155 mm/s**, **Y** = **0**, **yaw** = **−0.033 rad/s**, and **battery voltage** = **23.431 V**. The raw IMU scaling follows §7.4. **Flag ambiguity:** one firmware interface description states 0x01 = enabled, 0x00 = disabled, while a separate CAN feedback description states 0x00 = enabled. This conflict is not resolved here; inspect the actual firmware or test the bit against a measured motor-enable state before depending on it.

7.2 Ultrasonic feedback packet

Chassis configurations with ultrasonic sensors may output a separate **19-byte** frame:

Index	Description
0	Header 0xFA
1–12	Six signed 16-bit readings for ultrasonic A–F, in mm, MSB first
13–16	Four reserved bytes
17	XOR/BCC of bytes 0–16
18	Footer 0xFC

Ultrasonic feedback is only transmitted when the model is fitted with the applicable sensors. The documented configuration excludes standard LSR100 and standard LSR200 Series models from this optional telemetry.

7.3 Auto-charging feedback packet

When the optional docking kit is fitted, the controller sends an **8-byte** auto-charging packet:

Index	Description
0	Header 0x7C
1–2	Charging current, signed int16, **mA**, MSB first
3	Infrared detection flag
4	Charging-active flag (0 = not charging; nonzero = charging)
5	Reserved
6	XOR/BCC of bytes 0–5
7	Footer 0x7F

A nonzero IR flag indicates that the docking receiver has detected a station signal; this flag should not be interpreted as confirmation of battery current flow.

7.4 CAN status feedback

The 24-byte basic feedback described in §7.1 is divided into **three standard 8-byte CAN frames** on the documented CAN1 telemetry bus.

CAN ID	Bytes	Content
0x101	0–7	0x7B, enable/stop flag, X speed (2), Y speed (2), Z/yaw speed (2)
0x102	0–7	Accel X (2), Accel Y (2), Accel Z (2), Gyro X (2)
0x103	0–7	Gyro Y (2), Gyro Z (2), battery millivolts (2), BCC, 0x7D

All multi-byte fields are transmitted MSB first. The header/footer and additional XOR byte retain the serial-packet structure but are not required for CAN frame integrity because CAN provides native frame checking. The specified telemetry uses **three CAN IDs**, as listed here.

7.5 Wheel odometry and velocity reporting

The STM32 firmware computes chassis velocities from measured wheel states and vehicle geometry. For the documented differential-drive firmware branch, using the internal model codes (S100, S300Mini, S300), the C implementation uses the following structure:

```
Send_Data.Sensor_Str.X_speed =
  ((MOTOR_A.Encoder + MOTOR_B.Encoder) / 2) * 1000;
```

```
Send_Data.Sensor_Str.Y_speed = 0;
Send_Data.Sensor_Str.Z_speed =
   ((MOTOR_B.Encoder - MOTOR_A.Encoder) / Wheel_spacing) * 1000;
```

In the firmware MOTOR_A.Encoder and MOTOR_B.Encoder represent wheel linear speeds in **m/s**, and Wheel_spacing is the effective distance between driven wheel centers in **m**. The firmware reports linear velocities in mm/s and yaw rate scaled by 1000 from rad/s. This is **velocity feedback**, not a complete pose estimator or a guarantee of absolute accuracy. Wheel slip and geometry calibration matter.

7.6 IMU interpretation and scaling

The C63A's **ICM20948** provides accelerometer, gyroscope and magnetic-sensor capabilities. The chassis standard telemetry reports **three axes of accelerometer data and three axes of gyroscope data**, aligned with the robot coordinate convention. It does not follow that magnetometer measurements are included in the basic 24-byte frame.

The documented firmware settings are accelerometer range **±2 g** and gyroscope range **±500°/s**. Approximate SI conversion factors are:

Acceleration [m/s²] = signed_raw_acceleration / 1672
Angular rate [rad/s] = signed_raw_gyroscope / 3753

When the robot is stationary and level, the +Z accelerometer reading should be near **+9.8 m/s²** in the documented coordinate convention. The raw values are signed 16-bit integers. Always verify actual IMU full-scale settings after a firmware update, since any change would alter these conversion factors.

8. Internal Chassis Communication

8.1 Docking controller to main controller

The optional automatic charging receiver/controller (S23A controller) sends messages to **C63A over CAN at 500 kbit/s**. The documented motion/charge-status CAN ID is **0x182**:

Byte	Signal / meaning
0–1	X target speed for docking, signed 16-bit, mm/s, MSB first
2–3	Measured dock/charger voltage in its documented scaled representation
4–5	Z/yaw target motion value in the documented docking packet
6	Docking state and infrared-receiver bits
7	Charging-current encoded value

The state byte's bit 0 indicates docking active; bits 1–4 encode individual IR receiver flags R_A, R_B, L_A, and L_B. **Engineering units for bytes 2–3, 4–5 and 7 require verification against the installed S23A firmware** (e.g. charge-current field width/scaling and yaw-motion description); therefore this packet should be treated as a firmware-integration reference, **not** a validated third-party implementation specification. Get the matching S23A firmware protocol before direct modification.

8.2 Main controller to servo driver

For one documented CANopen motor-speed command, the C63A sends an SDO write to a servo driver. The example uses **standard CAN ID 0x601** for node 1 and a write to object **0x60FF, subindex 0x03**:

Byte	Value / field
0	CANopen SDO command byte 0x23
1–2	Object index 0x60FF in CANopen wire byte order (FF 60)
3	Subindex 0x03
4–5	First wheel-speed component
6–7	Second wheel-speed component

For actual SDO traffic, confirm numeric scaling, signed representation and low/high-byte ordering from the servo-driver object dictionary. This is an *internal controller-to-driver protocol*, distinct from the simpler host CAN command at 0x181.

8.3 Servo driver to main controller

An enabled mapped TPDO from a servo driver returns two four-byte motor measurement values in an **8-byte CAN frame**. The drive firmware uses IDs around **0x190 + NodeID − 1** for one configured TPDO mapping. A measurement is described as a wheel encoder/speed reading with the corresponding drive scaling (the reference also mentions 0.1 r/min). Do not parse the data as raw encoder increments unless that is confirmed by the active mapping and drive object dictionary.

9. Control Architecture and Firmware Tasks

9.1 Hardware communication architecture

A typical system has an onboard computer (for ROS and applications), LiDAR and optional perception devices, the C63A STM32 motion controller, and CAN-connected servo drives. The documented baseline connections are:

```
Remote PC / engineering workstation
    | network (SSH / ROS services)
Onboard computer (ROS applications; platform-dependent)
    | UART4 serial control / chassis telemetry
C63A STM32 controller -- I2C --> ICM20948 IMU
    | CAN (500 kbit/s)
    +-- Servo driver(s) --> Brushless hub motors
    +-- Optional charging controller / dock module
    | UART1: debug / firmware programming
    | UART2: optional app / Bluetooth communications
    | UART3: RS485 interface
    +-- Optional RC receiver / handheld control / ultrasound / RGB
```

The host normally handles LiDAR processing, SLAM/localization and path planning. The STM32 handles low-latency chassis control, drive communications, onboard sensor capture and state reporting. The specific ROS distribution and included navigation packages depend on the purchased computing option; they are not specified universally by this low-level manual.

9.2 MCU and FreeRTOS program organization

The controller uses **STM32F407VET6**, firmware written in **C**, a **Keil MDK-ARM / Keil 5** project and **FreeRTOS** task scheduling. Startup initializes peripherals, reads the model-selection potentiometer, selects the corresponding vehicle model and creates real-time tasks.

Scheduling group	Documented tasks / role
Startup/highest	StartInitTask: operating-system/hardware setup, then terminates
High priority	RobotControl_task: command arbitration, chassis kinematics and motion targets
High priority	HeartbeatTask: checks servo-driver connectivity
Medium priority	Sensor acquisition, ultrasonic data, IMU, serial communication, auto-charging and related control peripherals
Low priority	OLED/diagnostic reporting, app status display, RGB status and maintenance/user indications
Interrupt context	MCU interrupt handlers for UART/CAN and other peripherals execute outside normal task scheduling

A high-priority task may preempt a lower-priority task. Incorrect blocking logic or failed peripheral initialization can cause a control task to hang. Verify watchdog, timeout and task behavior against the delivered firmware rather than assuming that the example program offers certified functional safety.

9.3 Ultrasonic-avoidance flow

On installed ultrasonic-equipped models, periodic ultrasonic measurements update near-field obstacle state. When the optional avoidance mode is active, the local controller evaluates obstacles against commanded direction. Forward obstacles can trigger avoidance steering; reversing toward a rear sensor's obstacle can result in a stop. The action is accompanied by a repeated buzzer alert. This function can interfere with narrow-gap navigation and should be qualified within each application.

9.4 Automatic docking flow

Automatic charging involves coordination of **four functional software/firmware roles**: docking station, robot docking receiver, robot STM32 chassis controller and onboard ROS host. The station emits IR guidance; the receiver reports detection/status; the motion controller drives the chassis during the approach; and the host participates in high-level task or docking behavior. Full docking customization requires the matching firmware and docking-module interface package, not just the base-chassis UART command protocol.

10. STM32 Firmware Programming

10.1 General cautions

The C63A supports firmware download through the supported serial/USB bootloader path and through **SWD**. Reprogramming is a maintenance/development operation that can disable the chassis if the firmware image or model configuration is incorrect. Use the actual controller revision's firmware file and a correctly wired interface. Back up the existing configuration and confirm safe power supply conditions before any update.

10.2 Serial/USB programming

The serial programming method uses an auto-download path using a controller USB-to-UART interface, a corresponding **CH9102-class USB-to-TTL driver**, and **FlyMCU** firmware flashing software. Some model housings expose an external Type-C connector wired to Serial Port 1 for firmware programming; other models require access to the C63A PCB. The proper USB connector and cable type may vary with hardware version.

1. Connect the suitable USB data cable to the **designated programming connector**, not the regular host control UART4 connector.
2. Install the proper USB serial device driver and verify the COM/serial device appears without an error.

3. Start the compatible programmer, select the correct serial port, target and verified firmware image.
4. Apply the firmware configuration required for that controller revision and start programming.
5. If execution after flashing is enabled, wait for the controller to reboot and confirm the OLED model/hardware state before enabling motors.

Programming-tool settings vary by firmware and controller revision. Use the tool version and programming configuration provided for the delivered robot.

10.3 SWD programming

The controller supports an **ST-Link** programmer, or **J-Link** when configured appropriately. The SWD pin assignments are:

Programmer signal	MCU signal
SWDIO	PA13
SWCLK	PA14
GND	GND

Install the programmer driver, verify recognition in the development environment, connect the pins to the correct controller header, select the matching firmware project and download. The Keil project is configured for ST-Link by default; changing to J-Link requires programmer settings to be updated. Be mindful of shared ground, target supply and correct pin voltages.

11. Precautions and Maintenance

11.1 FreeRTOS tasks and code changes

An RTOS task can block other critical operations if it assumes a communication interface has been initialized when it has not. After a firmware change, test initialization order, queue behavior, motor-enable logic, failure recovery and communication timeouts. Hardware interrupts are executed outside normal task priorities. Always maintain a recoverable firmware image.

11.2 Controller power outputs

Note that although 5 V and 3.3 V power rails exist, these connectors cannot supply arbitrary external loads. Recommended upper loads are approximately **1.5 A on the 5 V output** and **200 mA on the 3.3 V output** for the general MCU-board outputs. A separate board-level supply path may feed the onboard Raspberry Pi/Orin-class computer. **Do not interpret an advertised separate 5 V/5 A port as the rating of every 5 V header**; check the physical connector, protection and actual power-distribution drawing.

11.3 Motor protection

Avoid extended motor stalling or operation with a blocked wheel. Continuous high temperature can damage magnets and bearings, while water ingress can corrode bearings and harm the motor assembly. Inspect wheel and bearing condition periodically and listen for unusual noise. Use low-speed tests on a secured rig after replacing a motor or driver.

11.4 Battery use and charging

The chassis OLED displays battery voltage. Charge promptly when battery voltage falls below approximately **20.0 V** for the documented battery arrangement; this is **not** a universal threshold for all optional battery packs. The battery includes overcharge and over-discharge protection. **Do not operate the robot while charging**. Keep the charging port and connector protected against accidental shorts and verify that the

charger is correct for the delivered battery chemistry, voltage and connectors. Manual and automatic charging accessories depend on the purchased configuration.

11.5 Interfaces to verify before deployment

Before integrating the robot in a live environment, confirm: ordered hardware options, correct model selection, actual wheel and track dimensions, all motor directions, host serial port selection, active feedback flag semantics, CAN bus configuration, link-loss stopping behavior, safety-stop function and available charging hardware. **Do not assume a ROS driver, full SLAM/navigation stack, STM32 source code, or proprietary commissioning software is included simply because it is referenced in a development manual.** Those items are subject to the delivered software package and sales agreement.

End of manual.